Promo Finder: finding promos in microdrama scripts

Kuku AI Generalist assignment · Problem 1 (Find the Promos) · Indra Sihag

Code: github.com/sihag06/kuku-promo-finder · App: Streamlit link · Raw outputs: Drive link · Video: Loom link

1. The problem I picked, and why

An editor watches every episode to pick promo moments, one show at a time, while new shows ship weekly. I picked this problem because it sits on Kuku's growth path (every promo is a paid acquisition ad), because the brief's rules make the output **checkable** (20–60 s, hook in 3 s, cliffhanger ending, understandable cold), and because the 50-episode pack let me **measure** quality instead of arguing it. The tool is meant to cut the editor's job from watching ~100 minutes per show to reviewing a ranked shortlist, not to replace the editor.

2. Assumptions (where the brief was silent)

- **Script time = final-cut time.** If not, align once per episode with subtitles and apply the offset.
- **Text only.** Visual hooks are judged from ACTION / CAMERA / SFX lines and flagged *visual_hook* for a footage check.
- **A promo** is 1–3 hard-cut segments totalling 20–60 s; cuts only on camera changes and never mid-line. Cold opens (a later hook shown first) and cross-episode joins are allowed.
- **Viewer:** a cold US scroller on TikTok/Reels/Shorts. The title already gives away the central twist, so that is fair game; late-arc resolutions count as spoilers. Sexual content, strong violence and threats to a child are flagged; a human decides.
- **Output:** every valid candidate plus a posting order. The tool assists; the editor makes the final pick.

3. How it works

```
scripts → parse → show bible → signals → mine moments → assemble → judge → validate → rank → editor review
 (.txt)    code     AI ×1/show     code      AI ×1/episode       code     AI ×1/clip    code       code     human (app)
```

> **Design rule: the AI never writes a timestamp.** It points at lines by id; code turns ids into times, snaps cuts to camera changes and enforces every rule. Invented ids are dropped. (Why it matters: GPT-5.4, asked to pick promos directly, broke the 20–60 s rule in 18 of its 69 picks.)

What I gave to AI: judgement only. One call per show builds a cast list that links each character's different labels (the pack introduces characters by description and names them later). One call per episode lists up to 6 hooks and 6 cliffhangers. One call per candidate judges it **seeing only the clip's own lines**, the honest version of "makes sense on its own": it first writes what a stranger would understand and what confuses them, then scores hook (first 3 s), cliffhanger (last 5 s), works-cold, emotion, spoiler risk and ad safety. **What I kept as code:** parsing, free keyword signals (slaps, reveals, open questions, episode endings), building four clip shapes, all rule checks, de-duplication, cost quotas and ranking. Ranking is a readable weighted formula in *config.yaml* with two outputs: a **quality rank** and a **posting order** (quality + early episodes first, since a new viewer lands on Episode 1, + a variety rule so the top 10 is not one scene five times). **Where a human stays in:** a Streamlit review screen shows each clip's exact lines; the editor accepts, rejects or notes a better cut.

4. Scale and cost

Measured on the 50-episode pack (gpt-5-mini)	Value	Lever
LLM calls / tokens	201 calls · 400k in / 309k out	hidden "thinking" tripled output vs my estimate → *reasoning: low*
Cost per show	$0.72 (reruns ≈ $0: every answer is cached)	judge cap (150 clips), per-episode quota, cheaper model
Time per show	~17 min at 4 parallel calls	10+ parallel calls on a paid key
Kuku-scale estimate	~10 shows × 60 episodes a week ≈ $9/week	self-hosted open model behind the same interface

Model swap is one line. Every call goes through one client; each stage's model is a *provider:model* string (OpenAI, Anthropic, Gemini, or any OpenAI-compatible server, so an API Kuku provides plugs straight in). Busy models fall back to alternatives, daily caps are detected, and keys pasted into the app are never stored. **Free tiers:** Gemini's free tier allowed only 20 requests a day for its stronger model, while one show needs ~200; a paid key is the honest requirement at volume. **Paid upgrade I would add:** a video-aware pass to check flagged visual hooks, and subtitle alignment for timing.

5. How I know it is good, and where it fails

References: (1) **15 promos I picked by hand**, blind, before any tool output, from 15 episodes (I excluded Episode 1, which I had studied while designing); (2) **26 promos from GPT-5.4**, a stronger model, picked by two different prompts and kept only where both agreed; (3) **an independent blind rater** (GPT-5.4, shuffled clips, no tool scores) for the top 20, as a stand-in for human ratings I did not have time to do. Matching: *loose* = right scene, cut differently; *strict* = same opening within 3 s and at least half the footage.

	Rules only (no AI)	AI pipeline
Promos breaking a brief rule	0 / 150	0 / 150
My 15 picks found anywhere in the list (loose)	9 (60%)	**12 (80%)**
My picks found in the top 20 (loose)	2 (13%)	**5 (33%)**
Top-20 promos matching a human or GPT-5.4 pick	12 / 20	**16 / 20**
Blind rater (GPT-5.4): top 20 rated 4+ out of 5 · mean	–	**13/20 (65%)** · 3.6

The evaluation changed the tool. It exposed three real flaws, each fixed and re-measured: one score mixed "is it good" with "post early episodes first" (now two separate ranks); strong hooks were paired with too-early cliffhangers, giving clips under 20 s that were rejected (now they extend to the next unresolved line); and de-duplicating before applying quotas silently discarded clips (now one pass).

Where it picks badly:

- **Late-season reveals.** Several of my picks were payoffs (a conspiracy unmasked, a deal revealed) that the cold-viewer judge ranks #90–#140: it cannot tell who is who. I chose them knowing the story. This is a real disagreement between "makes sense to a stranger" and "big reveal"; only an A/B test of posted promos can settle it.
- **Cost caps.** 3 of my 15 picks were never judged: only 150 clips (max 4 per episode) reach the paid judge. Raising the caps costs about $0.003 per extra clip, a choice for the editor.
- **Exact cuts rarely match** (0 strict matches in the top 20): the tool aims for ~35 s clips; mine average ~47 s and hold longer on reaction shots.
- **Without AI it fails badly:** the rules-only mode's #1 "cliffhanger" was a happy, resolved ending, because it ended an episode with fading music. The AI judge exists to catch exactly this.
- **Order inside the top 20 is not reliable.** The rater agrees the shortlist is mostly good, but its order and the tool's barely relate (Spearman ≈ –0.2 on 20 clips, i.e. no signal). The tool is good at *what* gets shortlisted, not at ranking near-equals, which is why an editor reviews the shortlist instead of auto-posting #1.
- **Stitched clips are the weakest.** The rater's lowest scores went to clips needing backstory and to cold-open / cross-episode joins whose jump "feels disconnected"; next step: penalise jumps that change scene or cast.
- **Text is not video:** a written slap may play weak on screen, so visual hooks are flagged, not trusted.

How I would know it worked at Kuku: the editor acceptance rate of the top 10 (logged in the app), minutes saved per show, and, for posted promos, 3-second hold rate, click-through and install-to-Episode-1 conversion against editor-picked promos in an A/B test.

6. AI tools I used, and where they got it wrong

Claude (Cowork) for architecture, code and tests; **GPT-5-mini** runs the pipeline; **GPT-5.4** builds the reference picks and rates blind; **Gemini** was tried first. **One place it got it wrong:** Claude wrote the reference-pick script as a plain loop, so 30 slow calls ran one after another with no progress output, and a 15-minute run looked frozen. I questioned why it was not parallel; it now uses the shared parallel runner with a live counter. Every other miss (a wrong data count, a regex that silently dropped all questions, the de-duplication order bug, a Gemini free-tier cap) is logged with its fix in *docs/ai_usage_log.md*.